

Predicting flight prices: the regression, step by step

How Farecast turns raw fare quotes into a daily price index, a seasonal forecast, a crude-oil pass-through estimate and a buy-or-wait decision. Every step below is implemented in `app/models.py`; numbers are measured on 158 routes / 496 route-airline series (2024-01-01 to 2026-09-15).

0. Notation and input

The only input is a table of observed quotes. One row is: a price seen on a given *search date*, for a flight departing on a given *flight date*, on one route flown by one airline.

Symbol	Meaning
s	search date - the day the price was observed
f	flight date - the day of departure
d = f - s	days to departure (the booking horizon)
i	a series: one route flown by one airline, e.g. SYD-LHR:QF
r(i)	the route of series i, e.g. SYD-LHR
P(i, s, f)	the quoted one-way fare

Two different things are called 'price' and must not be confused: the **quote** P depends on both when you look and when you fly; the **index** built in step 2 depends only on when you fly. Forecasting operates on the index; buy-timing operates on the quote.

1. The booking curve: separating *when you buy* from *when you fly*

A fare rises as departure approaches. If that effect is left in the data, a route observed mostly at short notice looks expensive for reasons that have nothing to do with the season. So the horizon effect is estimated first and divided out.

The estimate must not be confounded by *which* flights happen to be observed at each horizon - expensive peak-season flights are quoted more often near departure. The fix is flight fixed effects: compare a flight only against itself, by demeaning log fare within each (series, flight date) cell before averaging across flights.

Estimator

```
y(i, s, f) = log P(i, s, f)
y~(i, s, f) = y(i, s, f) - mean_s[ y(i, s, f) ]      # within-flight demeaning
c_b        = mean over all obs with d in bucket b of y~
C_r(d)     = exp( c_b(d) - c_b(30) )                # normalised: C_r(30) = 1
```

Horizons are grouped into 15 buckets, dense near departure and coarse far out (0-2, 2-4, ... 105-135, 135-181 days), because quotes thin out with horizon. Bucket effects are linearly interpolated between bucket midpoints, so C is defined for every integer d from 0 to 180. The curve is fitted per route where a route has at least 60 distinct flight dates, otherwise the pooled curve is used.

Why this matters for the advice

C is the only thing that decides *when to buy*. Its argmin is the recommended purchase horizon, and its shape differs sharply by haul: short-haul troughs around 46 days out, long-haul around 75. That difference is learned from data, not assumed.

2. The daily fare index

Every quote is divided by the booking-curve factor for its horizon, which removes the horizon effect and leaves a quantity comparable across booking windows. Quotes sharing a flight date are then aggregated, giving one number per series per calendar day - the 1-day-granularity series the forecast is built on.

```
Pn(i, s, f) = P(i, s, f) / C_r(d)                  # horizon-normalised
X(i, f)     = mean over s of Pn(i, s, f)           # daily index
X7(i, f)    = centred 7-day rolling mean of X(i, .) # rolling week
```

The rolling week is used for correlation and oil work, where day-to-day noise would swamp the signal; the regression itself is fitted on the unsmoothed daily index, so smoothing cannot leak future information into the fit.

3. Crude oil: first the lag, then the magnitude

Fuel is a large share of airline cost, but a change in crude does not reach fares immediately - carriers hedge, and fares are set in advance. The delay is estimated before the size of the effect, because the two need different estimators.

3a. Finding the lag

Brent daily spot is smoothed over 30 days and logged. Both oil and fares are differenced over 28 days, which removes the level and leaves co-movement. For each candidate lag from 0 to 90 days, oil is shifted forward and correlated with the fare-index changes; the lag maximising the mean correlation across series is selected.

```
O(t)      = log( 30-day rolling mean of Brent )
dO(t,L)   = O(t-L) - O(t-28-L)                # 28-day oil change, lagged by L
dX(i,t)   = log X7(i,t) - log X7(i,t-28)
L*        = argmax_L mean_i corr( dX(i,:), dO(:,L) )
```

Measured on the full sample: **L* = 42 days**, with per-series argmaxes averaging 40.6 days.

3b. Estimating the pass-through (and a trap)

The slope of that same change-on-change regression is a *biased* estimate of the elasticity: short-horizon fare changes are dominated by demand noise, which attenuates the coefficient - here by roughly a factor of three. Regressing the seasonal model's residuals on oil is also biased low, because within a single route one year of oil is collinear with the seasonal terms.

The unbiased route is Frisch-Waugh-Lovell: residualise both log fare and the oil term on each route's seasonal design, then pool. Identification comes from seasonal phases differing across routes while the oil path is common to all of them.

```
Z_r = seasonal design matrix for route r (see step 4)
ry  = log X - Z_r (Z_r'W Z_r)^-1 Z_r'W log X      # fare, seasonality removed
rx  = O(.-L*) - Z_r (Z_r'W Z_r)^-1 Z_r'W O(.-L*) # oil, seasonality removed
beta = sum_r _W / sum_r _W                        # pooled elasticity
```

Measured: **beta = 0.319** (d log fare per d log Brent). The synthetic data is generated with a known pass-through of 0.30, so this estimator recovers the truth; the change-on-change slope returns 0.367 on the same data, which is the attenuation described above.

No lookahead

Oil after the forecast origin is unknown in reality, so the oil series is frozen at its last observed value and carried forward flat. The model therefore never sees a future oil shock - visible in the results as negative bias in the split whose test year contains a price spike.

4. The forecast regression

With the horizon effect divided out and the oil effect held at a known coefficient, what remains to explain is the seasonal and weekly shape of the daily index. That is an ordinary least-squares regression of log index on a compact calendar design.

4a. Design matrix

Seasonality uses Fourier terms rather than month dummies: any smooth annual pattern is a sum of sine waves whose periods divide the year. Each harmonic contributes a sine/cosine pair, which lets the regression place a peak at any date ($a \cdot \sin + b \cdot \cos$ is one wave with fitted phase and height). Harmonic 1 gives a single annual peak and trough; harmonic 2 allows two peaks per year and asymmetric run-ups - the shape real fares have. Two harmonics is four parameters against eleven for month dummies, and the curve is smooth and defined on every calendar day.

```
u(f) = 2*pi * dayofyear(f) / 365.25
Z     = [ 1,  sin(u), cos(u),  sin(2u), cos(2u),  D_Tue..D_Sun ]
```

D_Tue..D_Sun are six day-of-week dummies (Monday is the reference), capturing the cheap-midweek / expensive-weekend pattern in the *departure* date.

A linear time trend was tested and **deliberately excluded**: on training windows of one to two years it fits noise and extrapolates it, costing about 3 percentage points of out-of-sample error. Seasonality plus the oil term carries the level movement instead.

4b. Fitting: oil offset, recency weights, route pooling

The oil contribution is subtracted from the left-hand side rather than fitted as a free column, so its coefficient is the FWL estimate from step 3b and cannot be re-estimated (badly) inside each short window.

$$y(i, f) = \log X(i, f) - \beta * (O(f-L^*) - \text{mean } O)$$

Observations are weighted by age with an exponential half-life, so recent seasons count more than old ones - a fare regime from several years ago should not carry the same weight as last year:

$$w(f) = 0.5^{(\text{age}(f) / H)}, \quad H = 730 \text{ days}$$

Finally, the seasonal fit is pooled by route. Airlines competing on one route share a demand pattern, so estimating it separately per airline spends data re-learning the same curve. One weighted fit per route is taken, then each airline gets a price-level offset on the intercept:

$$\begin{aligned} b_r &= (Z'WZ)^{-1} Z'W y_r && \# \text{ per-route seasonal fit, } W = \text{diag}(w) \\ a_i &= \text{mean}(y_i - Z b_r) && \# \text{ per-series level offset} \\ \log X^i &= Z b_r + a_i + \beta * (O(f-L^*) - \text{mean } O) \end{aligned}$$

Residual scale is the weighted standard deviation of the fit residuals; the reported interval is the 95% band $\exp(\log X^i \pm 1.96 \text{ sigma})$. Fitting is on the log scale throughout, which keeps fares positive and makes airline differences multiplicative rather than additive.

4c. Chosen by validation, not by taste

Every structural choice above was selected on a validation year and only then confirmed on a held-out test year (scripts/sweep.py, data/sweep_results.json):

Choice	Alternatives tested	Outcome
Linear trend	on / off	off - on costs ~3pp MAPE
Fourier harmonics	1 / 2 / 3	2 - 1 underfits, 3 adds nothing
Oil term	off / fixed-beta	on - ~1.3pp better
Recency half-life	none / 730 / 365 / 180 / 90 d	730 d - 90 d starves seasonality
Route pooling	per-series / per-route	per-route + level offsets
Index aggregation	median / mean / min	mean
Curve buckets	coarse / default / fine	no material difference
Ridge penalty	0 / 0.1 / 1 / 10 / 100	0 - no gain

5. From forecast to a buy-or-wait decision

The forecast above predicts the index for a *departure* date. The decision a traveller faces is different: given a specific flight, on which remaining day should it be bought? That comes from the booking curve, anchored to the price actually quoted today - so the model supplies the shape while the live quote supplies the level.

$$\begin{aligned} P^{\wedge}(\text{buy at horizon } d) &= P_{\text{now}} * C_r(d) / C_r(d_{\text{now}}) \\ d^* &= \text{argmin over } d \leq d_{\text{now}} \text{ of } C_r(d) && \# \text{ recommended buy horizon} \\ \text{save} &= P_{\text{now}} - P^{\wedge}(d^*) \end{aligned}$$

'Wait' is shown only when the predicted saving exceeds 3% of the current price; otherwise the verdict is 'book now'. For return trips both legs are priced on the same purchase date and the sum is minimised jointly, since a round trip is bought in one transaction.

6. Correlation between routes and airlines

Correlations are computed on 7-day differences of the log rolling-week index, not on levels. Two unrelated routes both have summer peaks, so their levels correlate strongly for a trivial reason; differencing removes the shared seasonal component and leaves genuine co-movement.

$$\begin{aligned} g(i, t) &= \log X7(i, t) - \log X7(i, t-7) \\ R &= \text{corr}(g), \text{ pairs with } \geq 120 \text{ overlapping days} \end{aligned}$$

Pair type	Mean correlation	Reading
Same route, different airline	0.817	carriers on a route track each other closely
Same airline, different route	0.058	little network-wide common pricing
Unrelated pairs	0.034	baseline

Measured across 122,760 pairs from 496 series. Competitive response within a route dominates; an airline's own pricing across different routes is close to independent.

7. Validation protocol

Accuracy is measured by temporal holdout. Everything that is fitted - the booking curve, the index normalisation, the oil lag and elasticity, and the regressions - is re-estimated using only data before the cutoff. The test period is then scored blind, and oil is frozen at the cutoff so no future information enters through that channel.

Two baselines are reported, because a forecast that cannot beat a naive rule is not worth running: **seasonal-naive** (same calendar date one year earlier, 364 days back to preserve weekday) and **train-mean** (the series average over the training window).

Metric	Train ≤ 2024-12-31 test 2025	Train ≤ 2025-12-31 test 2026
Forecast MAPE	7.75%	12.78%
Seasonal-naive MAPE	10.91%	13.1%
Train-mean MAPE	13.0%	14.16%
Bias	+2.48%	-9.95%
Series / daily observations	496 / 181,040	496 / 121,024

The second split's negative bias (-9.95%) is the frozen-oil assumption doing its job: its test year contains a crude spike the model could not have known about, so fares are systematically under-predicted. That is the honest cost of refusing lookahead, not a fitting error.

7b. Scoring the buy-timing advice

Price accuracy and decision quality are different questions, so the advice is scored separately. For each test flight the simulation stands at a standing point 30 days before that route's own curve trough, follows the recommendation, and pays the nearest quote actually observed on the recommended day - you cannot buy at a price nobody quoted.

The standing point is route-relative by necessity. A fixed 60-day anchor stands *past* the trough on long-haul routes, where the advice can only ever say 'buy now' and scores a trivial zero; that mistake made the measured capture ratio look like 0.31 rather than 0.71.

Outcome vs buying at the standing point	Test 2025	Test 2026
Mean saving	+2.94%	+3.03%
Median saving	+2.99%	+3.07%
Standard deviation	2.6 pp	2.64 pp
10th / 90th percentile	-0.34% / 6.25%	-0.33% / 6.34%
Flights helped / hurt	85.4% / 9.1%	85.1% / 9.1%
Share of the perfect-hindsight saving captured	71%	73%
Flights simulated	6,755	3,359

Broken out by curve shape: routes that bottom out late (> 60 days, mostly long-haul) yield 1.3% on average, while routes bottoming out within 60 days yield 3.31%. Worst observed single-flight outcome across both test years is about -5%: the advice only shifts a purchase by a few weeks, so the downside is bounded by drift over that window.

8. What this model does not do

- **It is calendar-driven, not event-driven.** Flash sales, capacity changes and strikes are not predicted; they appear as residual noise. Remaining error is dominated by the autocorrelated demand shocks the design deliberately does not chase.
- **The elasticity is a level relationship, not a causal claim.** Oil and fares share business-cycle drivers; the regression measures association after seasonality, not a fuel-cost mechanism.

- **Fare data here is synthetic.** No free source publishes daily route-airline fares: public fare statistics are quarterly or monthly averages, price-analysis APIs return quartiles rather than series, and continuous feeds are commercial. The generator's structure - booking curve, seasonality, route-level shocks, lagged oil pass-through - is known, which is what lets the estimators above be checked against a ground truth they were not told. Real quotes drop into the same schema without changing any model code.

- **Synthetic noise is smoother than real fares.** Real tails would be fatter than the bounded downside reported above.